

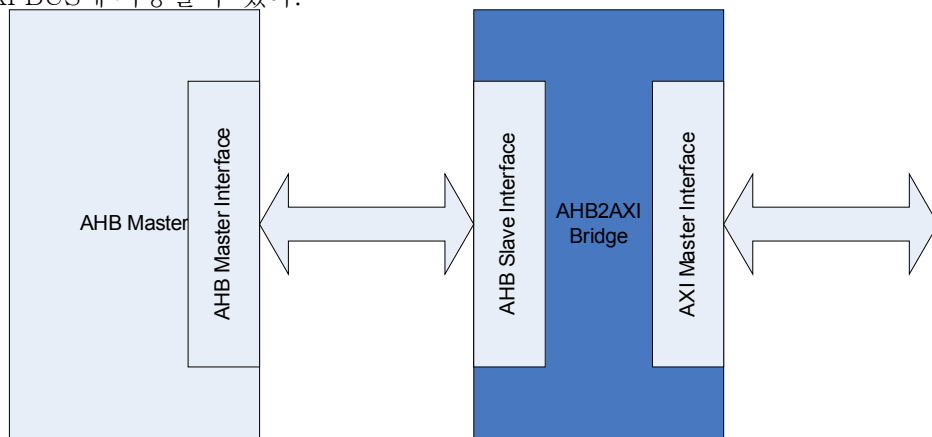
Title	AHB2AXI Bridge Integration Guide
Project	None
Description	AHB2AXI Bridge의 Integration을 위한 guide
Author	holelee
Date	22/May/2005

(*) AHB2AXI Bridge의 사용

AHB2AXI Bridge는 기존의 AHB Master IP나 AHB Platform에 AXI slave혹은 AXI Interconnection을 붙이기 위해 사용된다. 크게 다음과 같은 쓰임을 가질 수 있다.

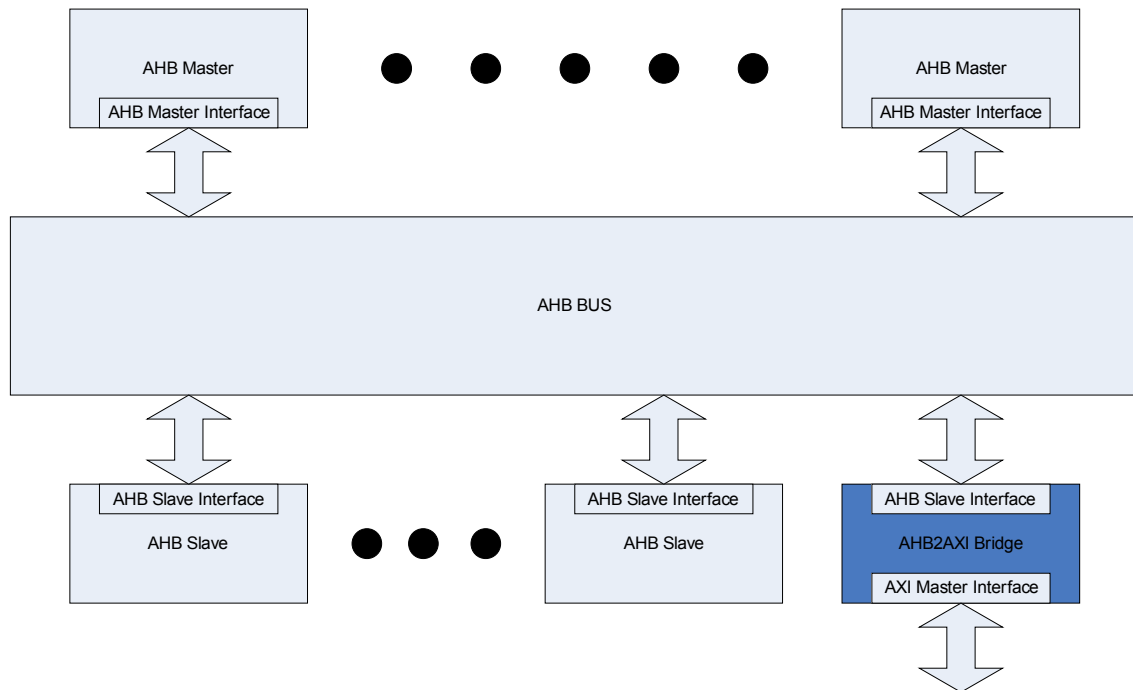
- AHB Master IP를 직접 붙이는 경우

AHB Master IP를 직접 AXI BUS로 연결하는 방법이다. ARM9과 같은 CPU를 이런 방법을 통해서 AXI BUS에 사용할 수 있다.



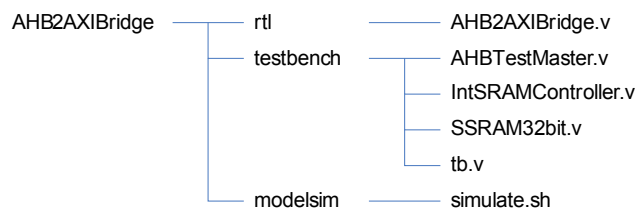
- AHB BUS를 붙이는 경우

AHB BUS의 한 slave로 AHB2AXI Bridge가 사용되는 경우이다. 이 경우에 여러개의 AHB Master가 AHB BUS를 통해서 AXI Slave, 혹은 AXI Interconnection에 접근할 수 있다.



(*) 제공되는 모듈 및 파일

AHB2AXI Bridge는 AHB2AXIBridge.v라는 단일 Verilog source로 제공된다. 그 module을 테스트하기 위해 AHBTestMaster.v IntSRAMController.v SSRAM32bit.v tb.v라는 verilog source code와 modelsim용 compile script인 simulate.sh를 제공한다. 아래와 같은 directory 구조로 되어 있다. AHBTestMaster.v는 RANDOM한 AHB Transaction을 수행하는 AHB-lite Master module이고, IntSRAMController.v는 AXI Internal SRAM Controller, SSRAM32bit.v는 Synchronous 32bit SRAM module이다. 전체 testbench의 top module은 tb.v이다.



(*) test 방법

testbench에서 test vector를 생성하는 AHBTestMaster.v는 초기에 BYTE/Halfword에 대한 transfer를 잠시 테스트한 후에 RANDOM read/write를 수행하는 식으로 구성되어 있다. RANDOM read/write는 INCR4 혹은 INCR을 이용하여 4-beat burst를 수행하도록 되어 있고, AHBTestMaster.v가 생성하는 AHB Bus transaction이 AHB2AXIBridge를 통해 AXI IntSRAMController를 지나 Synchronous 32bit SRAM인 SSRAM32bit.v에 읽고 쓰기를 진행

하게 된다. simulate.sh를 수행하면 자동으로 source code를 compile하고 simulation이 시작된다. 특별한 에러가 발생하지 않는 한 simulation은 끝나지 않고 계속 진행하게 된다.

(*) Signal 정리

AHB2AXIBridge.v에서 사용되는 입출력 신호들은 다음과 같다.

- Clock&Reset

Clock과 Reset은 AHB와 AXI에서 동일한 신호를 사용한다.

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
CLK	IN	Clock generator	Common clock signal for both AHB & AXI
RESETn	IN	Reset controller	Common active low reset signal for AHB & AXI

- AHB Signals

AHB 신호들은 다음과 같다. 일반적인 AHB Slave가 필요한 signal을 가진다. AHB2AXI Bridge는 SPLIT/RETRY를 지원하지 않으므로 그에 해당하는 port는 없다.

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
HADDR[31:0]	IN	AHB Master	AHB address
HTRANS[1:0]	IN	AHB Master	AHB transfer type
HWRITE	IN	AHB Master	AHB transfer direction
HSIZE[2:0]	IN	AHB Master	AHB transfer size(support up to 32bit)
HBURST[2:0]	IN	AHB Master	AHB burst type
HPROT[3:0]	IN	AHB Master	AHB protection control
HWDATA[31:0]	IN	AHB Master	AHB write data bus
HRDATA[31:0]	OUT	AHB Master	AHB read data bus
HREADY_IN	IN	Other Slaves	AHB transfer done
HREADY_OUT	OUT	AHB Master	AHB transfer done
HRESP[1:0]	OUT	AHB Master	AHB transfer response
HSEL	IN	AHB Decoder	AHB Slave select
HMASTLOCK	IN	AHB Arbiter	AHB Locked sequence indicator

- AXI Write Address Channel signals

ID tag을 가지지 않는다는 점만 제외하고는 AXI interface의 모든 signal을 지원한다.

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
AWADDR[31:0]	OUT	AXI Slave	AXI Write address

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
AWLEN[3:0]	OUT	AXI Slave	AXI burst length
AWSIZE[2:0]	OUT	AXI Slave	AXI burst size
AWBURST[1:0]	OUT	AXI Slave	AXI burst type
AWLOCK[1:0]	OUT	AXI Slave	AXI lock type
AWCACHE[3:0]	OUT	AXI Slave	AXI cache type
AWPROT[3:0]	OUT	AXI Slave	AXI protection type
AWVALID	OUT	AXI Slave	AXI write address valid
AWREADY	IN	AXI Slave	AXI write address ready

– AXI Write Data Channel signals

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
WDATA[31:0]	OUT	AXI Slave	AXI write data
WSTRB[3:0]	OUT	AXI Slave	AXI write strobe
WLAST	OUT	AXI Slave	AXI write last
WVALID	OUT	AXI Slave	AXI write valid
WREADY	IN	AXI Slave	AXI write ready

– AXI Write Response Channel signals

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
BRESP[1:0]	IN	AXI Slave	AXI write response
BVALID	IN	AXI Slave	AXI write response valid
BREADY	OUT	AXI Slave	AXI write response ready

– AXI Read Address Channel signals

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
ARADDR[31:0]	OUT	AXI Slave	AXI read address
ARLEN[3:0]	OUT	AXI Slave	AXI burst length
ARSIZE[2:0]	OUT	AXI Slave	AXI burst size
ARBURST[1:0]	OUT	AXI Slave	AXI burst type
ARLOCK[1:0]	OUT	AXI Slave	AXI lock type
ARCACHE[3:0]	OUT	AXI Slave	AXI cache type

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
ARPROT[3:0]	OUT	AXI Slave	AXI protection type
ARVALID	OUT	AXI Slave	AXI read address valid
ARREADY	IN	AXI Slave	AXI read address ready

- AXI Read Data Channel signals

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
RDATA[31:0]	IN	AXI Slave	AXI read data
RRESP[1:0]	IN	AXI Slave	AXI read response
RLAST	IN	AXI Slave	AXI read last
RVALID	IN	AXI Slave	AXI read valid
RREADY	OUT	AXI Slave	AXI read ready

(*) Integration 방법

우선 AHB-lite master에 직접 연결하는 법을 알아본다. 제일 먼저 고려할 것은 clock과 reset이다. AHB2AXI Bridge는 기본적으로 AHB와 AXI가 동일한 clock과 reset을 사용한다는 가정하에 설계되었다. reset은 반드시 동일해야 하고, Clock이 서로 다른 경우에 대해서는 AHB2AXI Bridge의 clock input port(CLK)에는 AHB clock을 연결해야 한다. AXI clock이 AHB보다 느린 경우에는 AHB2AXI Bridge의 AXI master interface에 별도로 설계된 AXI Downward synchronizing bridge를 연결하여 다른 속도의 AXI signal을 생성할 수 있다. 반대로 AXI clock이 더 빠른 경우에 대해서는 AXI upward synchronizing bridge를 연결하면 된다. AHB 관련 신호들은 AHB-lite master의 HADDR, HTRANS, HWRITE, HSIZE, HBURST, HPROT, HWDATA, HRDATA, HRESP 등을 그냥 AHB2AXI Bridge의 port에 연결하면 된다. AHB-lite master의 port중에 HLOCK이 있으면 그것을 HMASTLOCK port에 연결하고, 없으면 HMASTLOCK port에는 1'b0를 연결한다. AHB-lite master의 HREADY에 HREADY_OUT을 연결하고 HREADY_IN은 HREADY_OUT을 그냥 연결하면 된다. HSEL은 항상 1'b1로 연결한다.

일반적인 AHB master는 AHB-lite master가 가지고 있는 port에 HBUSREQ, HGRANT 등의 port를 더 가지고 있다. 일반적인 AHB master에 연결하는 경우에 대해서는 HBUSREQ는 open하고 HGRANT는 항상 1'b1로 연결하면 된다. 일반적인 AHB master의 HLOCK 신호는 AHB-lite master와 달리 arbitration 시점에서 유효하므로 그것을 latch하여 AHB2AXI Bridge의 HMASTLOCK port에 연결하여야 한다. 이 부분에 대해서는 AMBA AHB manual을 보고 직접 작성하도록 한다.

AHB BUS에 연결하는 방법은 훨씬 간단하다. 단순히 AHB BUS에 일반적인 slave를 연결하듯

이 그냥 연결하면 된다.

AXI쪽 interface는 ID tag을 사용하지 않는다는 점을 제외하고는 연결에 아무런 지장이 없을 것이다. 위에서 언급한 바대로 AXI BUS가 다른 clock으로 동작한다면 AHB2AXI Bridge에 Downward synchronizing bridge나 Upward synchronizing bridge를 연결한 후에 다시 AXI BUS에 연결하면 된다.